

Project Alpha — Checkpoint Update 2

Autonomous Drone Swarm · Implementation progress & locked decisions · May 25, 2026 (rev. geofence)

This update covers everything since Checkpoint 1: two core modules built and tested, the multi-agent tensor convention, and a series of architectural decisions on agent lifecycle, observation gating, and coordinate frames. It is a living reference — read alongside Checkpoint 1, not instead of it.

1. Progress Snapshot

Module	Status	Tests
coverage/coverage_map.py	DONE	21/21 passing
controller/mid_level_controller.py	DONE	16 pass + 2 known issues
policy/observation.py	Next	—
policy/reward.py	Pending	—
env/drone_env.py	Pending	—
train.py	Pending	—

Project relocated to ~/Drone/SwarmProject with a flat layout (env/ controller/ coverage/ policy/ configs/) — the earlier nested src/project_alpha/ structure was dropped as unnecessary for a single research project.

2. CoverageMap — Built & Tested

Tracks scanned cells across all parallel envs. Owns grid resolution, cell marking, and map queries. Does not own scan range (a drone property, passed in per call) or encoding strategy (lives in observation.py).

Multi-agent tensor convention (locked)

All query methods adopted a single flattened convention: internal grid stays per-env (num_envs, grid, grid); methods take env_ids (one entry per agent) plus per-agent positions, and return per-agent flattened output (num_agents_total, ...). env_ids[i] maps agent i to its env's shared grid. Reads never conflict; the only writer (mark_scanned) handles two agents in one env via uint8 index_put_ accumulation then OR into the grid.

Methods

Method	Purpose
pos_to_cell	continuous (x,y) → clamped (cell_x, cell_y) indices
mark_scanned	mark cells within scan_range of each agent (accumulate handles overlap)
get_full_map	per-agent copy of its env's flattened 10x10 map
get_local_window	fixed 11x11 window via adaptive pool — scale-invariant (Phase 2+ only)

get_coverage_pct	per-env (None) or per-agent (env_ids) coverage fraction
is_fully_covered	threshold check, per-env for done conditions

Optimization applied: `_batch_mask` preallocated in `__init__` and cleared with `zero_()` each call, removing per-step allocation churn.

Geometry note from testing: cell centers sit at half-cell offsets (2.5, 7.5, ...). A drone exactly on a 4-cell boundary (e.g. 25,25) is 3.54m from every nearby center — correctly marks nothing if `scan_range < 3.54m`. Confirmed correct, not a bug.

3. MidLevelController — Built & Tested

Bridge between RL policy and physics. Translates per-agent (dx, dy) actions into 3D velocity setpoints, owns the Z-axis entirely (policy never controls altitude), and runs the agent lifecycle state machine.

State machine (locked)

Mode	Meaning	Z height	Odometer	Velocity
ACTIVE (0)	scanning/traveling, policy-driven	SCAN_HEIGHT 5m	accumulates	toward action target
RETURNING (1)	rule-driven flight to base	RETURN_HEIGHT 15m	frozen	toward base
DOCKED (2)	arrived at base, terminal	—	frozen	zero
DEAD (3)	crashed/out-of-bounds, terminal	—	frozen	zero

Transitions: ACTIVE→RETURNING when odometer $\geq$ max_range $\times$ safety_factor; RETURNING→DOCKED on arrival within threshold; any→DEAD via mark_dead() for catastrophic failure only (NaN/velocity blowup, collision — NOT boundary contact, which is handled by the geofence clamp). DOCKED and DEAD are terminal until full episode reset.

Additions this session

- **MODE_DEAD = 3** — new terminal state for crashed/out-of-bounds drones.
- **seed_positions()** — env MUST call after reset() to seed prev_pos/target with real spawn positions, else the first odometer update adds a phantom ~35m jump from origin.
- **mark_dead(dead_mask)** — env flags crashes; controller only records.
- **Odometer freeze** — only ACTIVE drones accumulate distance (multiply by active mask); RETURNING/DOCKED/DEAD frozen.
- **Velocity zeroing** — extended from DOCKED-only to DOCKED-or-DEAD.
- **Geofence clamp** — arena_min/arena_max params added; target xy clamped into arena bounds before velocity. self.raw_target stores the pre-clamp target so reward.py can detect wall-pushing.

Known issues (tracked, deferred)

These print as informational in the test suite, not failures:

Reset jump — mitigated by seed_positions(); the env-side call is now a hard requirement in the reset contract.
Velocity overshoot — drone moves at full scan_speed even when the target is sub-metre away, causing potential oscillation near targets. Acceptable for now since the policy re-targets every step; fix (clamp speed to remaining distance) noted for if jitter appears in sim.

4. Architectural Decisions (Locked This Session)

The bulk of this session was design. The following are committed decisions, with rationale.

4.1 Action space & altitude

Action is permanently 2D: (dx, dy) direction, scaled by radius_cap. Z is never an RL action — it is a flight-phase parameter set by mode. This matches real autonomous mission planners (planner picks 2D waypoints, flight controller manages altitude) and keeps the policy simple across all phases. Phase 2+ could add (dx,dy,dz) with zero rework if variable scan altitude is ever needed.

4.2 Scan vs travel mode — deferred

TRAVEL/SCAN mode flag removed from the policy for Phase 0-1. Drones move at a constant scan_speed and always scan. Rationale: scanning is free when already traversing, and the only real distinction (speed vs coverage tradeoff) has marginal value for a single drone. Architecture keeps speed constants (SCAN_SPEED, MAX_SPEED, SCAN_SPEED_THRESHOLD) ready; Phase 2+ can reintroduce variable speed with automatic threshold-based scanning, adding one action dim.

4.3 Return trigger — odometer, not battery

Battery is fully out of Phase 0-1. Return is triggered purely by distance traveled ($\text{odometer} \geq \text{max_range} \times \text{safety_factor}$). Swapping in real battery modeling later is a one-line change. The earlier 'ghost mode' (returning drones invisible to others) was rejected as physically implausible — drones detect each other locally with no global state. Replaced by a visible RETURNING mode broadcast in observations; the policy learns to discount returning drones.

4.4 Two-mask split — actor vs critic

Actor masking and critic masking answer different questions and must be separate:

State	Actor (policy gradient)	Critic (joint state)
ACTIVE	train	include
RETURNING	mask (rule-based, not learned)	INCLUDE (still affects swarm value)
DOCKED	mask	include (safe-at-base is informative)
DEAD	mask	EXCLUDE (state is meaningless noise)

Key rule: mask the actor when the action was not a learned decision; mask the critic only when the state itself is noise. RETURNING diverges (real state, non-learned action) → critic keeps it, actor drops it. DEAD is the only state excluded from the critic — a returning drone still occupies space, carries data home, and relays maps, so hiding it from the critic would alias 'returning-with-data' against 'dead-data-lost'.

4.5 Observation gating — per-field, at env boundary

A dual-stage gate, both implemented in drone_env.py. **Front-end (observation gate):** the env overwrites observation fields the instant a drone changes state, keeping forward-pass shapes uniform. Gating is per-field, not blanket: RETURNING drones get scan-derived fields (local window) sentinel-filled (all-1.0 chosen convention) but keep live position/velocity/mode for the critic; DEAD drones are fully sentinel-filled. **Back-end (loss gate):** actor/critic loss masking in the PPO update zeroes gradients per the table in 4.4. You cannot fix an input-data

problem at the loss level alone — the bad state still flows through the forward pass and corrupts the value estimate, hence both gates.

4.6 No mid-episode respawn

DOCKED and DEAD are terminal for that agent until a full episode reset. Respawnning a depleted drone mid-episode into an 80%-covered map shatters the Markov property — identical ego observations would map to wildly different returns depending on respawn timestamp, preventing the critic from fitting a stable value baseline. Continuous operation, if ever needed, will use synchronized episode-level resets, not staggered per-agent respawns.

4.7 Out-of-bounds = geofence clamp, NOT death (revised)

This supersedes the earlier 'out-of-bounds → DEAD' note. A drone touching the boundary should not die — that is unrealistic (real drones have geofences) and gives a needlessly sparse, harsh signal. Instead: the controller clamps the target xy into the arena bounds so the drone physically cannot leave, and reward.py applies a small penalty when the policy's pre-clamp raw_target was outside (signal: 'you steered into the wall'). No termination. The clamp is deterministic, so out-of-bounds attempts are genuinely the policy's responsibility (exploration noise included) — fair to penalize, but mild. DEAD is now reserved strictly for catastrophic failure: NaN/inf physics state, velocity blowup, and drone-drone collision (Phase 1+). The DEAD machinery is unchanged; it simply is no longer triggered by boundary contact.

4.8 Rectangular arena (committed)

The geofence clamp is rectangular (arena_min/arena_max), not polygonal. Non-rectangular survey sites are handled by over-covering with a padded rectangle — which is how real survey missions are planned anyway — and, if ever needed, a valid-cell mask in CoverageMap for coverage accounting. Arena shape is a coverage-accounting concern, not a where-can-the-drone-fly concern, so it stays out of the controller. arena_min/arena_max (rather than a single arena_size scalar) lets non-square rectangles and future origin shifts work with the same clamp code at no cost.

5. Env-Local Coordinate Contract (Invariant)

Isaac Lab clones environments with per-env world offsets stored in `env_origins` (`num_envs, 3`). Drone positions reported by the sim are in world coordinates including that offset. CoverageMap and the controller both assume clean env-local coordinates in `[0, arena_size]`.

INVARIANT: everything downstream of `drone_env.py` works in env-local coords.

```
drone_env.py converts once, before calling CoverageMap or controller:  
local_pos = world_pos - env_origins[env_ids]
```

Decision: corner-origin arena (local range `[0, 50]`), so the offset is a plain subtraction with no half-arena shift. Base station, `prev_pos`, and all targets must use this SAME frame — never mix world and local.

Consequence already designed in: `pos_to_cell` clamps out-of-range positions. This is safe only because the env guarantees valid local coords upstream. A clamped out-of-arena position would otherwise produce hallucinated edge-cell telemetry — which is exactly why RETURNING/DEAD drones get their scan-derived observation fields gated (4.5) rather than computed from a clamped position.

6. Phase 2+ Pins (Do Not Forget)

Per-agent belief maps. The current shared per-env coverage grid is a Phase 0-1 simplification that assumes instant, perfect map sharing. Under communication-range limits, each agent maintains its own belief map built only from what it scanned plus gossip from drones it actually met. `get_full_map` and `get_coverage_pct` become per-env-per-agent; grid shape goes `(num_envs, grid, grid) → (num_envs, num_agents, grid, grid)`. True global coverage (for done conditions) stays separate from agent-perceived coverage. The current `env_ids`-indexing convention is the deliberate stepping stone — signatures barely change (`index [env_ids, agent_ids]` instead of `[env_ids]`).

- **get_local_window optimization** — precompute the offset meshgrid in `__init__` when the method is activated (currently dead code, zeroed in Phase 0-1 obs).
- **Variable speed + threshold scanning** — reintroduce when sim-to-real realism is needed (4.2).
- **Battery modeling** — replaces the odometer return trigger (4.3).
- **Heterogeneous drones** — different `scan_range/max_range` per drone; `max_range` becomes a per-agent tensor. Not in same run for now.
- **Dock altitude** — decide whether docking at 15m is final or drones descend to ground (`DOCK_HEIGHT` currently unused).

7. Next Session — Implementation Order

1. `policy/observation.py` — Per-agent flattened obs assembly. Builds in the coordinate contract (5) and per-field gating (4.5). Env passes clean state (`pos, vel, yaw` extracted from quaternion); `observation.py` stays pure assembly.
2. `policy/reward.py` — Single reward function (mode-aware split deferred with TRAVEL/SCAN). Coverage delta, redundancy penalty, time, collision, milestone, safe-return, plus wall-push penalty using `controller.raw_target` vs clamped target (ACTIVE drones only).
3. `env/drone_env.py` — DirectRLEnv subclass. Owns world→local conversion, crash detection (calls `mark_dead`), reset contract (`reset → read spawn → seed_positions → coverage reset`), and the two gates.

4. `train.py` — SKRL MAPPO training loop for single drone, Phase 0.

Reset contract (must be honored by `drone_env.py`):

```
def _reset_idx(env_ids):
    controller.reset(env_ids) # clear mode/odometer
    spawn = read_drone_positions(env_ids) # world coords from sim
    local = spawn - env_origins[env_ids] # → env-local
    controller.seed_positions(local, env_ids) # prevent reset jump
    coverage_map.reset(env_ids)
```